ĐẠI HỌC QUỐC GIA HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA ĐIỆN TỬ - VIỄN THÔNG

------ΘΘΘ------

BÁO CÁO ĐỒ ÁN

XỬ LÝ TÍN HIỆU SỐ

THIẾT KẾ BỘ EQUALIZER KÊNH KHÔNG DÂY ỨNG DỤNG NHÂN CHẬP & BIẾN ĐỔI Z

THẦY ĐẶNG LÊ KHOA

THẦY NGUYỄN THÁI CÔNG NGHĨA

ĐẶNG BÁ TRẦN TRUNG – 23207121
NGUYỄN ĐỨC SƠN – 23207100
LÊ QUANG THÔNG – 23207113
GIAO HỮU LỰC – 23207080
LÊ HOÀNG QUỐC UY – 22207101

TP. HỒ CHÍ MINH – 07/2025

MỤC LỤC

ĐẠI HỌC QUỐC GIA HỒ CHÍ MINH 1
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN 1
KHOA ĐIỆN TỬ - VIỄN THÔNG 1
MỤC LỤC 3
DANH MỤC HÌNH 5
CODE CELL 6
LỜI CẢM ƠN 7
CHƯƠNG 1: GIỚI THIỆU & ĐẶT VẤN ĐỀ 8
1.1. Đặt vấn đề 8
1.2. Mục tiêu của bài báo cáo 8
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT 9
2.1. Môi trường lập trình 9
2.1.1. Anaconda Navigator 9
2.1.2. Jupyter Notebook 9
2.1.3. Thư viện hỗ trợ xử lý tín hiệu số trong Python 10
2.2. Khái quát về lý thuyết tín hiệu 11
2.2.1. Hệ thống tuyến tính bất biến thời gian (LTI) 11
2.2.2. Nhân chập (Convolution) 11
2.2.3. z – Transform 12
2.2.4. Bộ cân bằng tín hiệu (Equalizer) 12
2.2.5. Quy trình xử lý tín hiệu 13
CHƯƠNG 3: THỰC NGHIỆM Ý TƯỞNG 15
3.1. Hướng thực hiện ý tưởng 15
3.2. Lập trình Python trên Jupyter Notebook 16
3.3. Phân tích chương trình 20
3.4. Phân tích kết quả 21
CHƯƠNG 4: MỞ RỘNG ỨNG DỤNG CỦA CHƯƠNG TRÌNH 23
4.1. Giới thiệu 23

4.2. Lý thuyết về DCT – Discrete Cosine Transform 23
4.3. Thực nghiệm hóa 25
4.4. Lập trình Python trên Jupyter Notebook 25
CHƯƠNG 5: PHÂN TÍCH MSE & ĐÁNH GIÁ HIỆU QUẢ 31
5.1. Khái niệm và cách tính MSE 31
5.2. Lập trình Python trên Jupyter Notebook để tính MSE 31
CHƯƠNG 6: KẾT LUẬN & PHÁT TRIỂN 34
6.1. Tóm tắt bài báo cáo 34
6.2. Những điểm còn hạn chế & đề xuất phát triển 35
6.2.1. Hạn chế của chương trình hệ thống 35
6.2.2. Đề xuất hướng phát triển 35
6.3. Tổng kết 35
TÀI LIỆU THAM KHẢO 37

DANH MỤC HÌNH

Hình 1. 1. Tín hiệu gốc 21
Hình 1. 2. Tín hiệu sau khi qua hệ thống có white noise 21
Hình 1. 3. Tín hiệu sau khi khôi phục 21

Hình 2. 1. Công thức DCT thuận 1 chiều 23
Hình 2. 2. Công thức DCT thuận 2 chiều 24
Hình 2. 3. Công thức DCT nghịch 2 chiều 24
Hình 2. 4. Minh họa nâng cao chất lượng ảnh xám 25
Hình 2. 5. Kết quả 29
Hình 2. 6. Dung lượng ảnh sau khi nén 30

Hình 3. 1. Công thức tính MSE 31
Hình 3. 2. Kết quả tính toán MSE 33

CODE CELL

Code cell 1. 1. Các thư viện cần thiết 16
Code cell 1. 2. Hàm biến đổi thuận..... 16
Code cell 1. 3. Hàm biến đổi ngược..... 17
Code cell 1. 4. Tín hiệu đầu vào 17
Code cell 1. 5. Tín hiệu đầu vào 18
Code cell 1. 6. Thêm white noise..... 18
Code cell 1. 7. Áp dụng biến đổi z..... 18
Code cell 1. 8. Vẽ biểu đồ tín hiệu..... 19

Code cell 2. 1. Các thư viện cần dùng 26
Code cell 2. 2. Chuẩn hóa ảnh gốc..... 26
Code cell 2. 3. Hàm DCT..... 27
Code cell 2. 4. Tái tạo ảnh theo phần trăm 27
Code cell 2. 5. Tái tạo ảnh với 3 mức độ 28
Code cell 2. 6. Hiển thị kết quả ảnh đã xử lý..... 29

Code cell 3. 1. Các thư viện cần thiết 31
Code cell 3. 2. Hàm tính MSE 31
Code cell 3. 3. So sánh mảng..... 32
Code cell 3. 4. Tính MSE giữa hai tín hiệu 32
Code cell 3. 5. Tính MSE giữa hai ảnh..... 32

LỜI CẢM ƠN

Lời đầu tiên, cho phép chúng em bày tỏ lòng biết ơn vô cùng sâu sắc đến Thầy Đặng Lê Khoa và Thầy Nguyên Thái Công Nghĩa vì đã tận tâm hướng dẫn đồ án này cho nhóm chúng em. Hai thầy đã hướng dẫn bằng tất cả tâm huyết để chúng em có thể đạt được thành công trong việc hoàn thành đồ án môn học một cách trọn vẹn nhất.

Nhóm chúng em xin bày tỏ lòng biết ơn sâu sắc đến các Thầy/ Cô thuộc khoa Điện tử - Viễn thông đã tạo mọi điều kiện thuận lợi giúp đỡ nhóm chúng em trong quá trình học tập và nghiên cứu.

Nhóm chúng em cũng xin cảm ơn các anh chị và các bạn học đã góp phần hỗ trợ tối đa nhóm chúng em trong quá trình thực hiện đồ án môn học.

Nhóm chúng em rất mong nhận được sự góp ý và chỉ bảo tận tình từ quý Thầy/ Cô để nhóm chúng em có thể hoàn thiện kiến thức hơn trong tương lai.

Trân trọng!

CHƯƠNG 1: GIỚI THIỆU & ĐẶT VẤN ĐỀ

1.1. Đặt vấn đề

"Tam sao thất bản" – câu thành ngữ ám chỉ hiện tượng của một thông tin, lời nói bị sai lệch. Khi được truyền miệng qua nhiều người. Mỗi người có thể hiểu khác đi, vô tình thêm thắt, bớt đi hoặc diễn đạt không chính xác, dẫn đến thông tin cuối cùng không còn đúng như ban đầu.

Hiện tượng này tương tự với quá trình truyền tín hiệu trong hệ thống, khi tín hiệu được truyền qua nhiều kênh, bị tác động bởi môi trường và tính chất riêng của từng hệ thống. Kết quả là tín hiệu đầu ra cuối cùng có thể bị méo mó, nhiễu, không giống so với tín hiệu ban đầu mà ta cần truyền đi.

1.2. Mục tiêu của bài báo cáo

Để xử lý hiện tượng méo mó, nhiễu của tín hiệu đầu ra sau khi trải qua nhiều hệ thống, nhóm chúng em sẽ thiết kế ra một bộ Equalizer (hay còn gọi là bộ lọc ngược – Inverse Filter). Tác dụng của bộ Equalizer này là khôi phục tín hiệu đầu ra giống với tín hiệu ban đầu một cách chính xác nhất có thể, nhằm đảm bả thông tin nhận được không bị sai lệch do ảnh hưởng của hệ thống truyền dẫn.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Môi trường lập trình

2.1.1. Anaconda Navigator

Anaconda là phần mềm miễn phí cung cấp bộ công cụ phục vụ cho nghiên cứu và khoa học. Cài đặt Anaconda cho phép truy cập các môi trường lập trình Python hoặc R, được gọi là IDEs. IDE giúp phát triển mã nguồn dễ dàng, tương tự như Word hay Google Docs nhưng mạnh mẽ hơn, hỗ trợ viết, chỉnh sửa, gỡ lỗi mã, hiển thị dữ liệu, lưu biến và trình bày kết quả.

Hình 0.1. Logo Anaconda Navigator

Mỗi IDE có ưu nhược riêng, nhưng không ảnh hưởng đến cú pháp Python. Ngoài ra, Anaconda còn đi kèm hàng loạt thư viện Python được đóng gói sẵn, dễ dàng cài đặt và cập nhật. Đây là cách đơn giản nhất để sử dụng Python, thư viện, IDE chỉ trong một nền tảng. Theo trang chủ, Anaconda là cách dễ nhất để thực hiện khoa học dữ liệu và học máy (machine learning) với Python hoặc R trên một máy tính cá nhân.

2.1.2. Jupyter Notebook

Jupyter Notebook là một IDE chạy trên trình duyệt web và được khởi động từ Anaconda Navigator. Mỗi khối mã có thể được chạy riêng, giúp tăng sự linh hoạt và dễ thử nghiệm. Nó cho phép sử dụng nhiều loại văn bản trong cùng một Notebook như mã nguồn, kết quả, hình ảnh, phương trình và văn bản thường, rất hiệu quả để trình bày tài liệu và kết quả một cách rõ ràng, đẹp mắt. Vì chạy trên web, nó dễ chia sẻ và phù hợp để làm việc nhóm.

Hình 0.2. Logo Jupyter

JupyterLab là phiên bản mở rộng của Jupyter Notebook, được tích hợp thêm Terminal, Console và Text Editor. Tuy nhiên, nên bắt đầu với Jupyter Notebook vì dễ sử dụng và chứa đầy đủ công cụ cơ bản cho việc tìm hiểu về Python và phân tích dữ liệu.

2.1.3. Thư viện hỗ trợ xử lý tín hiệu số trong Python

Dưới đây là một số thư viện cơ bản và được dùng nhiều trong việc phân tích và xử lý trên tín hiệu số trong lập trình Python:

- **numpy**: Giúp xử lý số học hoặc thao tác trên mảng. Thư viện này hỗ trợ tạo tín hiệu rời rạc *x(n), h(n)* và thực hiện các phép toán như cộng, nhân, dịch thời gian.
- **matplotlib**: Hỗ trợ vẽ tín hiệu. Dùng để trực quan hóa tín hiệu theo thời gian, biểu diễn so sánh tín hiệu trước và sau khi xử lý hoặc đưa qua bộ lọc.
- **scipy.signal**: Thư viện hỗ trợ xử lý tín hiệu số. Cụ thể, module scipy.signal giúp thực hiện các phép toán tín hiệu như nhân chập, thiết kế bộ lọc, biến đổi tín hiệu và phân tích hệ thống.

2.2. Khái quát về lý thuyết tín hiệu

2.2.1. Hệ thống tuyến tính bất biến thời gian (LTI)

Trong xử lý tín hiệu số, hệ thống tuyến tính bất biến thời gian (LTI) là hệ thống tiêu chuẩn, phổ biến và dễ dàng phân tích nhất. Hệ thống này phải thỏa mãn hai điều kiện sau:

1) **Tuyến tính**: đầu ra của hệ thống phản ứng tuyến tính với đầu vào.

$$x_1(n) \rightarrow y_1(n)$$
$$x_2(n) \rightarrow y_2(n)$$
$$\Rightarrow ax_1(n) + bx_2(n) = ay_1(n) + by_2(n)$$

2) **Bất biến thời gian**: nếu đầu vào bị trễ một lượng thời gian, thì đầu ra cũng bị trễ một lượng tương ứng.

$$x(n) \rightarrow y(n)\delta x(n-k) \rightarrow y(n-k)$$

Vì hệ thống thỏa mãn tính chất tuyến tính và bất biến theo thời gian (LTI), nên đầu ra của hệ thống có thể được tính bằng *phép nhân chập* giữa tín hiệu đầu vào và đáp ứng xung.

2.2.2. Nhân chập (Convolution)

Nhân chập là một phép toán giúp xác định đầu ra *y(n)* của một hệ thống khi biết đầu vào *x(n)* và đặc trưng của hệ thống *h(n)*. Nó hoạt động bằng cách trượt hàm đáp ứng *h(n)* của hệ thống qua tín hiệu đầu vào *x(n)*, nhân từng cặp giá trị tại mỗi thời điểm rồi cộng lại. Quá trình này tạo ra một tín hiệu đầu ra thể hiện cách mà hệ thống làm thay đổi tín hiệu đầu vào theo đặc trưng của nó. Phép chập được dùng rất nhiều trong xử lý tín hiệu và ảnh:

$$x(n) \otimes h(n) = (x \otimes h)(n) = \int_{-\infty}^{+\infty} x(k)h(n-k)$$

Ví dụ: Cho tín hiệu *x(n)* = {**<u>2</u>**, 3, 4}; *h(n)* = {1, **<u>2</u>**, 3} → *y(n)* = {2, **<u>7</u>**, 16, 17, 12}

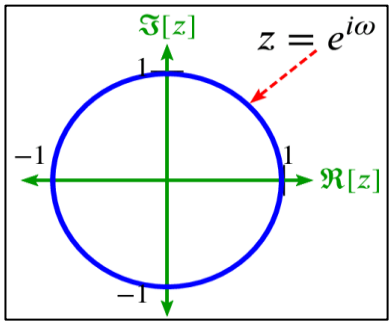
2.2.3. z – Transform

Biến đổi z là một công cụ toán học được dùng để biểu diễn và phân tích các tín hiệu, hệ thống rời rạc trong miền thời gian bằng cách chuyển sang miền tần số phức, chuyển từ miền thời gian *x(n), h(n)* sang miền *X(z), H(z)*. Nhờ các tính chất đặc trưng của *biến đổi z*, các phép toán như nhân chập trở nên đơn giản hơn, thuận tiện cho việc tính toán. Điều này giúp cho quá trình phân tích và thiết kế hệ thống trở nên dễ dàng và trực quan hơn.

$$X(z) = \sum_{-\infty}^{+\infty} x(n) z^{-n}$$

Tinh chất quan trọng được dùng trong bài báo cáo này của z – Transform sẽ là tính chất chuyển đổi *"Nhân chập trong miền thời gian sẽ là nhân thường trong miền tần số"*:

$$y(n) = x(n) \otimes h(n) \xrightleftharpoons{z-transform} Y(z) = X(z) \times H(z)$$



Hình 0.3. Đường tròn đơn vị

2.2.4. Bộ cân bằng tín hiệu (Equalizer)

Equalizer – Bộ cân bằng tín hiệu hay còn được gọi với cái tên khác phù hợp với mục đích sử dụng trong đồ án này chính là bộ lọc ngược (inverse filter).

Hình 0.4. Bộ Equalizer thực tế

Trong đồ án này, bộ cân bằng tín hiệu là một hàm *w(n)* có tác dụng khôi phục tín hiệu đầu ra giống với tín hiệu ban đầu cho vào hệ thống, giúp tăng tính chính xác, chuẩn chỉnh và rõ nét cho thông tin nhận được. Bộ lọc ngược w(n) được thiết kế sao cho khi áp dụng lên tín hiệu đầu ra *y(n)*, kết quả thu được *x'(n)* gần giống với tín hiệu đầu vào ban đầu *x(n)*. Nói cách khác, nó có vai trò *hủy bỏ ảnh hưởng của hệ thống h(n)* lên tín hiệu.

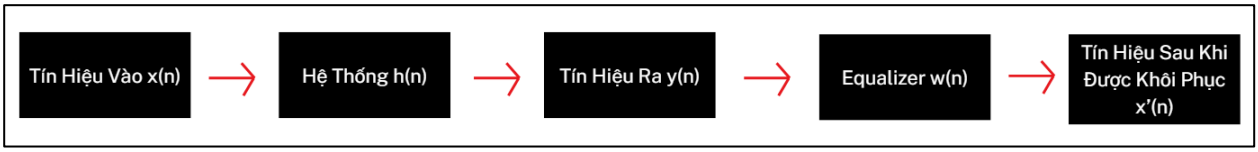
Bộ cân bằng tín hiệu kênh không dây được dùng để loại bỏ sự méo mó cũng như các tín hiệu gây nhiễu của điện xoay chiều, các thiết bị chuyển đổi trong một hệ thống,… Từ đó ta có thể khôi phục lại chính xác tín hiệu đầu ra sau khi đi qua các hệ thống.

ƯU ĐIỂM	NHƯỢC ĐIỂM
Cải thiện chất lượng truyền thông	Phụ thuộc vào đặc trưng của hệ thống
Tín hiệu khôi phục gần giống tín hiệu ban đầu	Không hoàn toàn giống với tín hiệu ban đầu
Dễ xử lý trong miền z	Phức tạp trong thiết kế hệ thống thực tế

2.2.5. Quy trình xử lý tín hiệu

Hệ thống nhận được tín hiệu vào *x(n)*, truyền qua một đáp ứng của hệ thống *h(n)* sẽ cho ra một tín hiệu *y(n)*. Tuy nhiên tín hiệu sẽ *y(n)* bị méo do nhiễu và biến dạng bởi đặc trưng riêng của hệ thống cũng như môi trường xung quanh. Để khắc phục vấn đề đó, ta cho tín hiệu

y(n) đi vào bộ lọc ngược *w(n)* để loại bỏ đi những ảnh hưởng đó. Kết quả thu được tín hiệu *x'(n)* gần giống với tín hiệu vào *x(n)*.



Hình 0.5. Các bước xử lý tín hiệu

CHƯƠNG 3: THỰC NGHIỆM Ý TƯỞNG

3.1. Hướng thực hiện ý tưởng

– Bước 1: *Truyền tín hiệu qua hệ thống*

Tín hiệu đầu vào x(n) được truyền qua hệ thống có đáp ứng h(n) và kết quả thu được tín hiệu đầu ra:

$$x(n) \otimes h(n) = y(n) \quad \textbf{(1)}$$

– Bước 2: *Khôi phục bằng bộ tín hiệu ngược*

Để loại bỏ những ảnh hưởng của hệ thống, cần cho *y(n)* đi qua bộ lọc ngược *w(n)*, thu được tín hiệu sau lọc:

$$y(n) \otimes w(n) = x'(n) \simeq x(n) \quad \textbf{(2)}$$

– Bước 3: *Trình bày theo ý nghĩa toán học*

Từ **(1)** và **(2)** suy ra:

$$x(n) \otimes h(n) \otimes w(n) = x'(n) \simeq x(n)$$

Chọn xung dirac δ(n) vì khi cho tín hiệu qua δ(n) sẽ không làm thay đổi tín hiệu, ta có:

$$x(n) \otimes \delta(n) = x'(n) \simeq x(n)$$

$$\Leftrightarrow h(n) \otimes w(n) = \delta(n) \quad \textbf{(3)}$$

– Bước 4: *Áp dụng biến đổi z*

Theo tính chất của biến đổi z, phép nhân chập ở miền thời gian tương ứng với phép nhân đại số ở miền z. Tuy nhiên, việc thực phép nhân chập trực tiếp ở miền thời gian sẽ rất dễ dẫn đến

sai số do phức tạp về mặt tính toán và độ dài tín hiệu, vậy nên cần biến đổi z để việc tính toán trở nên dễ dàng và chính xác hơn.

$$H(z) \times W(z) = 1 \rightarrow W(z) = \frac{1}{H(z)}$$

– Bước 5: *Tìm w(n)*

Sau khi có được W(z), ta dùng biến đổi z ngược để tìm ra w(n):

$$W(z) \xrightarrow{Inverse-z} w(n)$$

3.2. Lập trình Python trên Jupyter Notebook

– Thêm các thư viện cần thiết:

```
import numpy as np

import matplotlib.pyplot as plt
```

Code cell 1. 1. Các thư viện cần thiết

– Định nghĩa hàm biến dổi z (z – Transform) và biến đổi z ngược (inverse z – Transform):

```
def z_transform(x):

    w = np.arange(-2*np.pi, 2*np.pi, 4*np.pi/512)

    z = np.exp(1j * w)

    Xz = np.zeros(len(z), dtype = complex)

    for n in range(len(x)):

   Xz += x[n] * (z ** -n)

    return z, Xz
```

Code cell 1. 2. Hàm biến đổi thuận

```
def inverse_z_transform(Xz, N):

    w = np.arange(-2*np.pi, 2*np.pi, 4*np.pi/512)

    z = np.exp(1j * w)

    x_rec = []

    for n in range(N):

        xn = (1/(2 * np.pi)) * np.sum(Xz * z**(n) * (4*np.pi/512))

        x_rec.append(np.real(xn))

return np.array(x_rec)
```

Code cell 1. 3. Hàm biến đổi ngược

– Tạo tín hiệu đầu vào:

```
fs = 10

ts = 50

t = np.linspace(0, ts, fs*ts, endpoint = False)



f1 = 0.05

a1 = 1

phi_1 = np.pi / 4

s1 = a1 * np.sin(2*np.pi*f1*t + phi_1)
```

Code cell 1. 4. Tín hiệu đầu vào

```
f2 = 1

a2 = 0.5

phi_2 = np.pi / 2

s2 = a2 * np.sin(2*np.pi*f2*t + phi_2)

x = s1 + s2  # tín hiệu đầu vào
```

Code cell 1. 5. Tín hiệu đầu vào

– Hệ thống h(n) và nhiễu trắng:

```
h = [0.5, -1, 0.3]

y = np.convolve(x, h, mode = "same")

noise = 0.08 * np.random.randn(len(x))

y_noisy = y + noise
```

Code cell 1. 6. Thêm white noise

– Áp dụng biến đổi z và khôi phục tín hiệu:

```
z, H = z_transform(h)

W = 1/H

w_n = inverse_z_transform(W, fs*ts)



x_rec = np.convolve(y_noisy, w_n, mode = "same") #khôi phục
```

Code cell 1. 7. Áp dụng biến đổi z

– Vẽ biểu đồ trực quan hóa tín hiệu:

```
plt.figure(figsize=(12, 9))



plt.subplot(3, 1, 1)

plt.plot(t, x)

plt.title("Tín hiệu gốc x(n)")

plt.ylabel("Biên độ")

plt.xlabel("Thời gian")
```

```
plt.subplot(3, 1, 2)

plt.plot(t, y_noisy)

plt.title("Tín hiệu sau khi qua hệ thống + nhiễu: y(n)")

plt.ylabel("Biên độ")

plt.xlabel("Thời gian")
```

```
plt.subplot(3, 1, 3)

plt.plot(t, x_rec)

plt.title("Tín hiệu sau khi khôi phục x’(n)")

plt.ylabel("Biên độ")

plt.xlabel("Thời gian")

plt.tight_layout()
```

Code cell 1. 8. Vẽ biểu đồ tín hiệu

3.3. Phân tích chương trình

Về cách chọn đầu vào x(n): nhóm chọn tín hiệu sóng sin để có thể dễ dàng quan sát được sự thay đổi giữa tín hiệu vào x(n), tín hiệu sau khi qua hệ thống y(n) và tín hiệu sau khi được khôi phục x'(n)

Về cách chọn đáp ứng xung hệ thống h(n): mặc dù trong thực tế, kênh truyền h(n) mang tính ngẫu nhiên mình chưa được biết trước và thay đổi theo môi trường. Tuy nhiên, để mô phỏng và xử lý bằng thuật toán số, nhóm chọn một hàm h(n) cụ thể và hợp lý nhưng vẫn phải đảm bảo:

- Tín hiệu không bị quá yếu vẫn giữ được thông tin quan trọng của tín hiệu gốc.
- Tổng các hệ số h(n) đủ lớn nhưng vẫn ổn định và hữu hạn tránh gây sai lệch lớn khi khôi phục.

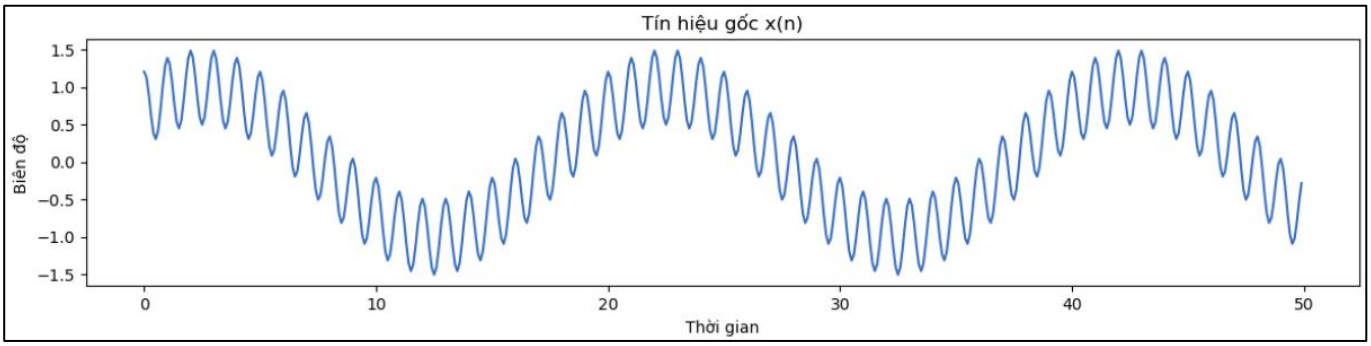
Cách hoạt động của chương trình tương tự với phần [**2.2.5. Quy trình xử lý tín hiệu**]

- Tạo tín hiệu vào *x(n)*
- Cho truyền qua hệ thống *h(n)*
- Cộng thêm nhiễu trắng để đồng bộ với điều kiện ảnh hưởng bởi môi trường
- Biến đổi z để tính Equalizer
- Biến đổi z ngược để thu được bộ lọc *w(n)*
- Dùng *w(n)* để khôi phục *x'(n)*

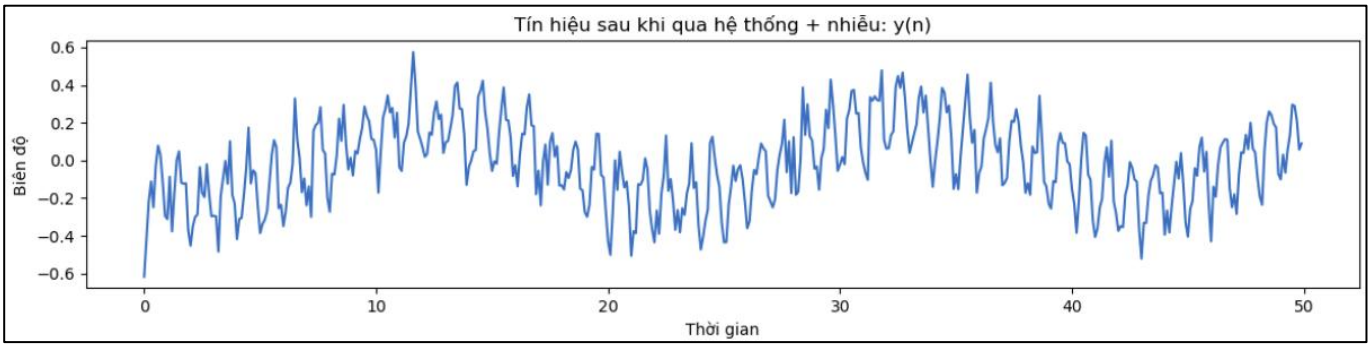
Các yếu tố mở rộng trong chương trình:

- Tín hiệu đầu vào được tạo ra bằng cách cộng hai sóng sin khác tần số, lệch pha. Điều này gần với thực tế hơn trong tín hiệu âm thanh và sóng truyền thông.
- Xây dựng hàm biến đổi z và biến đổi z ngược: không dùng thư viện có sẵn, nhóm xây dựng lại từ định nghĩa lý thuyết, giúp linh hoạt áp dụng cho các hệ thống khác nhau, tăng tính học thuật và tùy chỉnh cao.
- Ngoài ảnh hưởng của hệ thống, nhóm tạo thêm nhiễu trắng ngẫu nhiên là nhiễu từ môi trường để bám sát với thực tế, giúp kiểm chứng khả năng khôi phục trong điều kiện không lý tưởng.

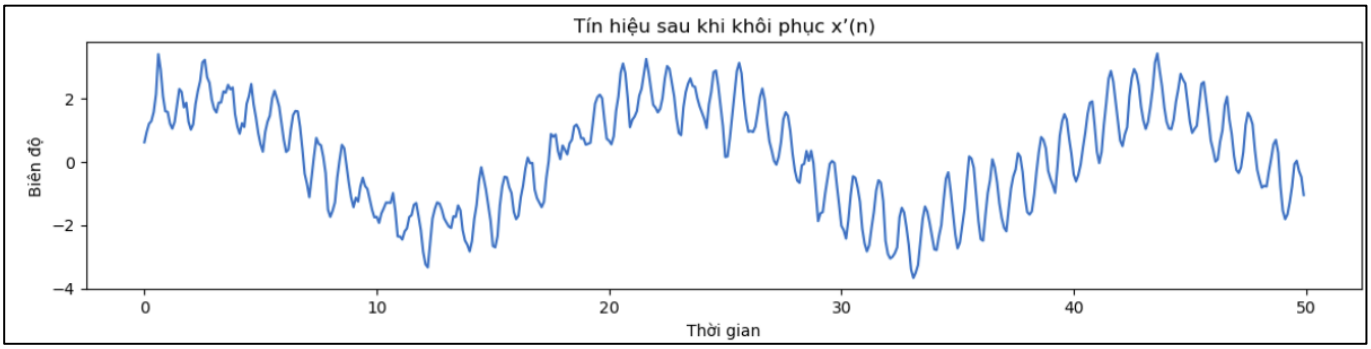
3.4. Phân tích kết quả



Hình 1. 1. Tín hiệu gốc



Hình 1. 2. Tín hiệu sau khi qua hệ thống có white noise



Hình 1. 3. Tín hiệu sau khi khôi phục

Hình 1: Tín hiệu gốc x(n)

- Đây là tín hiệu đầu vào được tổng hợp từ hai sóng sin, s1 (tần số thấp – biên độ lớn) và s2 (tần số cao – biên độ nhỏ).
- Tín hiệu có dạng tuần hoàn, đối xứng, nhiễu không xuất hiện.
- Thể hiện thông tin chuẩn được truyền đi qua hệ thống.

Hình 2: Tín hiệu sau khi qua hệ thống + nhiễu trắng → y(n)

- Tín hiệu bị méo bởi tác động của hệ thống, không còn giữ nguyên biên độ và hình dạng so với x(n)
- Nguyên nhân là do tín hiệu vào đã nhân chập với h(n) = [0.5, -1, 0.3] – một bộ lọc có tính chất làm suy giảm một số tần số nhất định, đặc biệt là tần số cao.
- Nhiễu trắng làm xuất hiện các đỉnh nhọn nhỏ, chớp nhoáng và phân bố ngẫu nhiên.
- Tín hiệu bị lệch khỏi đường sóng chính, gây mất pha nhẹ.
- Tất cả điều này mô phỏng đúng với tình huống thực tế trong truyền thông không dây là luôn có nhiễu.

Kết luận, việc sử dụng Equalizer là cần thiết để tái lập tín hiệu.

Hình 3: Tín hiệu sau khi khôi phục x'(n)

- Dạng sóng đã tái tạo gần đúng với x(n) về tổng thể: giữ được cấu trúc chu kỳ, đặc trưng tần số thấp (f1 = 0.05 Hz) và tần số cao (f2 = 1 Hz), đồng thời pha của tín hiệu cũng gần trùng với tín hiệu.
- Về biên độ, một số điểm cố thể lớn hơn so với x(n), do quá trình đảo ngược phổ trong lọc ngược có thể khuếch đại các tần số bị suy giảm trước đó. Tuy nhiên, biên độ vẫn ổn định ở mức cho phép.
- Nhiễu trắng đã giảm rõ rệt: tín hiệu lọc trở nên mượt, rõ ràng và dễ phân tích

Kết luận: kết quả trên cho thấy bộ lọc đã khôi phục hiệu quả dạng sóng, biên độ, tần số và pha. Điều này chứng tỏ nhóm đã thực hiện đúng mục tiêu thiết kế bộ Equalizer khôi phục tín hiệu.

CHƯƠNG 4: MỞ RỘNG ỨNG DỤNG CỦA CHƯƠNG TRÌNH

4.1. Giới thiệu

Trong chương 3, nhóm đã thiết kế và thực hiện bộ lọc ngược (Equalizer) để khôi phục tín hiệu x(n) từ tín hiệu đầu ra y(n) bị méo và nhiễu. Phương pháp này dựa trên việc phân tích hệ thống bằng biến đổi Z và thiết kế bộ lọc ngược w(n) sao cho x'(n) ≈ x(n).

Tuy nhiên, để mở rộng phạm vi tìm hiểu của bài học này, nhóm quyết định tìm hiểu bổ sung phần nội dung về DCT: Image Compression. Tương tự như xử lý tín hiệu sóng, ảnh cũng có thể bị mất mát thông tin trong quá trình truyền hoặc nén. Việc phục hồi ảnh sau nén bằng DCT (Discrete Cosine Transform) có chung bản chất đó là biến đổi sang miền tần số, xử lý và khôi phục lại.

Ngoài ra, để đánh giá hiệu quả khôi phục, nhóm sử dụng chỉ số MSE trong cả hai trường hợp: Equalizer tín hiệu và phục hồi ảnh từ DCT. Qua đó, nhóm liên kết hai hướng xử lý tín hiệu và ảnh dưới cùng một mục tiêu đó là khôi phục lại tín hiệu gốc sau khi bị biến dạng.

4.2. Lý thuyết về DCT – Discrete Cosine Transform

DCT (Discrete Cosine Transform) – Biến đổi cosin rời rạc là một kỹ thuật toán học dùng để chuyển đổi tín hiệu từ miền không gian (ảnh gốc) sang miền tần số. DCT giúp phân tách thành phần năng lượng của tín hiệu thành các hệ số mang thông tin tần số, sau đó giữ lại những thành phần quan trọng như thông tin chính (tần số thấp) và loại bỏ các thành phần ít quan trọng như nhiễu (tần số cao).

Việc loại bỏ các tần số cao giúp tiết kiệm dung lượng dữ liệu đáng kể mà không ảnh hưởng nhiều đến chất lượng hình ảnh. Đây là nền tảng của thuật toán nén ảnh JPEG.

$$C(u) = \alpha(u)\sum_{x=0}^{N-1} f(x)\cos\left[\frac{\pi(2x+1)u}{2N}\right]$$

Hình 2. 1. Công thức DCT thuận 1 chiều

$$C(u,v)=\alpha(u)\alpha(v)\sum_{x=0}^{N-1}\sum_{y=0}^{N-1}f(x,y)\cos\left[\frac{\pi(2x+1)u}{2N}\right]\cos\left[\frac{\pi(2y+1)v}{2N}\right]$$

Hình 2. 2. Công thức DCT thuận 2 chiều

$$f(x,y)=\sum_{u=0}^{N-1}\sum_{v=0}^{N-1}\alpha(u)\alpha(v)C(u,v)\cos\left[\frac{\pi(2x+1)u}{2N}\right]\cos\left[\frac{\pi(2y+1)v}{2N}\right]$$

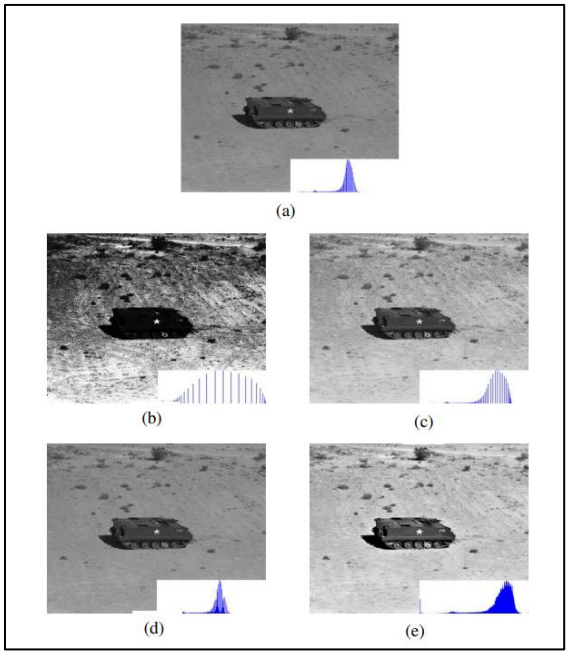
Hình 2. 3. Công thức DCT nghịch 2 chiều

Ưu điểm:

- Kết quả DCT là số thực, không phải số phức nên thuận tiện cho việc tính toán.
- DCT tập trung hầu hết năng lượng vào hệ số đầu (tần số thấp).
- Các hệ số sau (tần số cao) sẽ được loại bỏ để tiết kiệm dung lượng nhưng vẫn giữ được chất lượng hình ảnh.

Ứng dụng thực tế của DCT:

- Gửi ảnh qua mạng: nén ảnh bằng DCT giúp giảm kích thước ảnh, rút ngắn thời gian truyền tải và tiết kiệm dung lượng lưu trữ.
- Lưu trữ ảnh: DCT là nền tảng của chuẩn JPEG – định dạng ảnh phổ biến nhất hiện nay.
- Truyền thông số: giúp kích thước khung hình.



Hình 2. 4. Minh họa nâng cao chất lượng ảnh xám

4.3. Thực nghiệm hóa

Quy trình xử lý ảnh bằng DCT bao gồm các bước:

1) Đọc ảnh đầu vào, chuyển sang ảnh xám (grayscale) để giảm độ phức tạp và dễ phân tích.
2) Chuẩn hóa kích thước ảnh về 256×256 để đảm bảo tính nhất quán.
3) Thực hiện biến đổi DCT thuận, chuyển ảnh từ miền không gian sang miền tần số.
4) Giữ lại các hệ số tần số thấp (chứa thông tin chính), loại bỏ tần số cao (chứa chi tiết nhỏ và nhiễu).
5) Thực hiện DCT ngược để tái tạo lại ảnh, với mức phần trăm giữ lại được tùy chọn (ví dụ: 10%, 30%, 60%,…).
6) Hiển thị ảnh tái tạo và lưu lại để so sánh chất lượng, đồng thời kiểm tra dung lượng ảnh sau nén.

4.4. Lập trình Python trên Jupyter Notebook

– Thêm các thư viện cần thiết:

```
import numpy as np

import matplotlib.pyplot as plt

from scipy.fftpack import dct, idct

from skimage.io import imread

from skimage.color import rgb2gray

from skimage.transform import resize
```

Code cell 2. 1. Các thư viện cần dùng

– Đọc ảnh gốc và chuẩn hóa:

```
image = imread("anhgoc.jpg")

image = rgb2gray(image)

image = resize(image, (256, 256))
```

Code cell 2. 2. Chuẩn hóa ảnh gốc

– Hàm DCT 1 chiều và 2 chiều:

```
def dct2(a):

return dct(dct(a.T, norm='ortho').T, norm='ortho')



def idct2(a):

return idct(idct(a.T, norm='ortho').T, norm='ortho')
```

Code cell 2. 3. Hàm DCT

– Tái tạo ảnh gốc và giữ lại theo phần trăm:

```
image_dct = dct2(image)

def reconstruct_from_dct(dct_img, retain_fraction):

      rows, cols = dct_img.shape

      mask = np.zeros_like(dct_img)

      mask[:int(rows * retain_fraction), :int(cols *
retain_fraction)] = 1

      dct_filtered = dct_img * mask

return idct2(dct_filtered)
```

Code cell 2. 4. Tái tạo ảnh theo phần trăm

– Tái tạo ảnh với ba mức độ:

```
recon_10 = reconstruct_from_dct(image_dct, 0.1)

recon_30 = reconstruct_from_dct(image_dct, 0.3)

recon_60 = reconstruct_from_dct(image_dct, 0.6)
```

Code cell 2. 5. Tái tạo ảnh với 3 mức độ

– Hiển thị ảnh đã được xử lý với DCT:

```
plt.figure(figsize=(12, 8))



plt.subplot(2, 2, 1)

plt.imshow(image, cmap='gray')

plt.title("Ảnh gốc")

plt.axis('off')
```

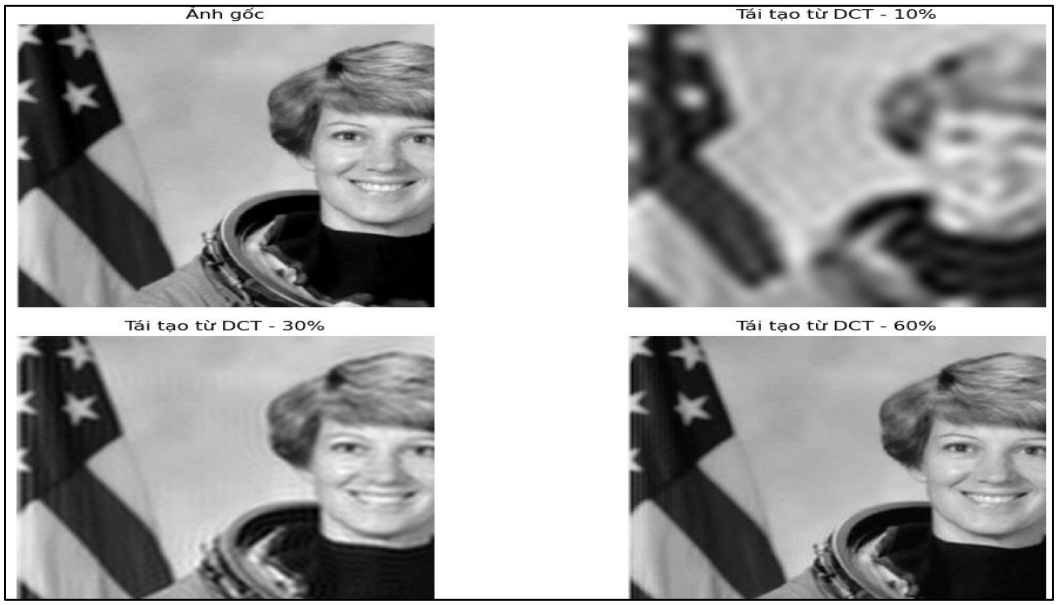
```
plt.subplot(2, 2, 2)

plt.imshow(recon_10, cmap='gray')

plt.title("Tái tạo từ DCT - 10%")

plt.axis('off')
```

```
plt.subplot(2, 2, 3)

plt.imshow(recon_30, cmap='gray')

plt.title("Tái tạo từ DCT - 30%")

plt.axis('off')
```

```
plt.subplot(2, 2, 4)

plt.imshow(recon_60, cmap='gray')

plt.title("Tái tạo từ DCT - 60%")

plt.axis('off')
```

Code cell 2. 6. Hiển thị kết quả ảnh đã xử lý

– Kết quả thử nghiệm:



Hình 2. 5. Kết quả

anhgoc	7/13/2025 2:46 PM	JPG File	26 KB
anh0.6	7/13/2025 2:48 PM	JPG File	22 KB
anh0.3	7/13/2025 2:48 PM	JPG File	20 KB
anh0.1	7/13/2025 2:49 PM	JPG File	15 KB

Hình 2. 6. Dung lượng ảnh sau khi nén

Sau khi ảnh được tái tạo từ DCT, ta nhận thấy: mức giữ lại hệ số càng cao thì ảnh càng sắc nét và gần giống ảnh gốc, đồng thời dung lượng lưu trữ cũng tăng lên. Vì vậy, cần cân nhắc chọn mức giữ lại (retain %) phù hợp để đảm bảo chất lượng ảnh đủ dùng, nhưng vẫn tiết kiệm được dung lượng – đặc biệt quan trọng khi lưu trữ hoặc truyền ảnh qua mạng.

CHƯƠNG 5: PHÂN TÍCH MSE & ĐÁNH GIÁ HIỆU QUẢ

5.1. Khái niệm và cách tính MSE

MSE (Mean Squared Error) là đại lượng đo lường mức độ sai khác giữa hai tín hiệu hoặc giữa các tệp ảnh với nhau. Dựa vào MSE ta có thể đánh giá được chất lượng khôi phục, tái tạo và nén tín hiệu hoặc ảnh khi MSE càng thấp thì chất lượng khôi phục càng tốt.

Công thức tính MSE:

Mean Squared Error (MSE) between two images such as $g(x,y)$ and $\hat{g}(x,y)$ is defined as [6]

$$\mathrm{MSE} = \frac{1}{MN}\sum_{n=0}^{M}\sum_{m=1}^{N}\left[\hat{g}(n,m) - g(n,m)\right]^2 \quad (1)$$

From Equation (1), we can see that MSE is a representation of absolute error.

Hình 3. 1. Công thức tính MSE

5.2. Lập trình Python trên Jupyter Notebook để tính MSE

– Thêm các thư viện:

```
import numpy as np

import matplotlib.pyplot as plt
```

Code cell 3. 1. Các thư viện cần thiết

– Hàm tính MSE:

```
def calculate_mse(signal1, signal2):

    signal1 = np.array(signal1)

    signal2 = np.array(signal2)
```

Code cell 3. 2. Hàm tính MSE

```
if signal1.shape != signal2.shape:

    print("Hai mảng không cùng kích thước")

    mse = np.mean((signal1 - signal2) ** 2)

return mse
```

Code cell 3. 3. So sánh mảng

– Tính MSE giữa tín hiệu gốc *x(n)* và tín hiệu khôi phục *x'(n)*:

```
mse_z = calculate_mse(x, x_rec)

print("MSE giữa x(n) và x'(n) (z -  transform):", mse_z)
```

Code cell 3. 4. Tính MSE giữa hai tín hiệu

– Tính MSE giữa ảnh gốc và ảnh được xử lý DCT:

```
mse_10 = calculate_mse(image, recon_10)

mse_30 = calculate_mse(image, recon_30)

mse_60 = calculate_mse(image, recon_60)

print("\nMSE giữa ảnh gốc và ảnh 10%:", mse_10)

print("\nMSE giữa ảnh gốc và ảnh 30%:", mse_30)

print("\nMSE giữa ảnh gốc và ảnh 60%:", mse_60)
```

Code cell 3. 5. Tính MSE giữa hai ảnh

– Kết quả tính toán:

```
MSE giữa x(n) và x'(n) (Z-transform): 1.5589379755497257

MSE giữa ảnh gốc và ảnh 10%: 0.005745645383983517
MSE giữa ảnh gốc và ảnh 30%: 0.0010742993831898142
MSE giữa ảnh gốc và ảnh 60%: 0.00014442668966819824
```

Hình 3. 2. Kết quả tính toán MSE

Kết quả MSE với tín hiệu sóng (z - transform) = 1.5589. Dù giá trị MSE cao nhưng nếu xem xét cụ thể thì sóng có biên độ từ ± 1.5 đến $\pm$ 2. Vì vậy kết quả MSE trên là một giá trị hoàn toàn có thể chấp nhận được và phù hợp với loại tín hiệu thời gian có nhiễu thực tế.

Kết quả MSE với ảnh tái tạo bằng DCT:

- + Mức giữ tần số thấp 10%: 0.00575 - ảnh khôi phục bị mờ, mất chi tiết nhỏ.
- + Mức giữ tần số thấp 30%: 0.00107 - ảnh rõ hơn, giữ được nhiều chi tiết hơn.
- + Mức giữ tần số thấp 60%: 0.00014 - gần như khôi phục hoàn toàn ảnh gốc.

Dải giá trị pixel của ảnh sau khi chuyển sang ảnh xám nằm trong đoạn [0,1], nên MSE nhỏ hơn 0.01 sẽ cho chất lượng ảnh tốt. Nhưng không thể so sánh trực tiếp MSE giữa phương pháp biến đổi z và phương pháp DCT vì:

- Biến đổi z dùng cho tín hiệu 1 chiều (sóng), trong khi DCT dùng cho tín hiệu 2 chiều (ảnh).
- Dải giá trị đầu vào khác nhau.
- Việc mất thông tin ở ảnh là chủ động (loại bỏ tần số cao).

Vì vậy nên xem xét, cân nhắc chọn MSE trong từng ngữ cảnh và mục tiêu xử lý riêng.

CHƯƠNG 6: KẾT LUẬN & PHÁT TRIỂN

6.1. Tóm tắt bài báo cáo

Trong đồ án lần này, nhóm đã nghiên cứu thành công mô hình xử lý tín hiệu số qua việc khôi phục lại tín hiệu đầu vào đã bị biến dạng (méo, nhiễu) do đặc trưng riêng của hệ thống và điều kiện môi trường bằng việc chế tạo Equalizer (bộ lọc ngược) sử dụng biến đổi z. Các công việc được tóm tắt lại như sau:

- Tạo sóng đầu vào x(n) được tổng hợp từ hai sóng sin và cho vào hệ thống h(n) giả lập nhưng vẫn mang tính thực tế cao vì có hàm nhiễu trắng.
- Khôi phục được tín hiệu x'(n) gần giống tín hiệu x(n) ban đầu.
- Xây dựng được hàm biến đổi z thuận - ngược vì hai hàm này không có sẵn trong thư viện.

Ngoài ra, nhóm đã mở rộng phạm vi báo cáo bằng cách tìm hiểu thêm về DCT: Image Compression – một kỹ thuật trong việc tái tạo ảnh. Các công việc được tóm tắt lại như sau:

- Tìm hiểu về DCT (Discrete Cosine Transform).
- Giữ lại tần số thấp (chứa các thông tin chính), loại bỏ đi tần số cao (chứa nhiễu, chi tiết nhỏ). Từ đó chủ động trong việc từ bỏ thông tin, tiết kiệm được dung lượng và thời gian đáng kể trong quá trình truyền tải nhưng vẫn giữ được chất lượng ảnh ở mức tốt.
- Sử dụng đại lượng MSE để xác định sai số giữa tín hiệu/ảnh sau khi được khôi phục so với bản gốc.

Các kết quả đạt được trong báo cáo:

- Tạo ra bộ Equalizer hoạt động ổn định trong việc làm giảm nhiễu, khôi phục được dạng sóng có hình dạng, biên độ, tần số, pha gần giống tín hiệu ban đầu muốn truyền đi.
- Sử dụng tốt các tính chất của biến đổi z trong việc xử lý hệ thống tuyến tính bất biến thời gian.
- Khôi phục được ảnh với nhiều mức tái tạo để xem xét, chọn lựa ảnh với chất lượng tốt nhưng vẫn tiết kiệm được dung lượng và thời gian trong quá trình truyền tải thông qua phương pháp DCT.

- Phân tích được những sai số giữa ảnh tái tạo so với ảnh gốc nhờ sử dụng MSE.

6.2. Những điểm còn hạn chế & đề xuất phát triển

6.2.1. Hạn chế của chương trình hệ thống

Những điểm hạn chế của đồ án:

- Hệ thống h(n) giả lập và tuyến tính bất biến, chưa phản ánh được đặc trưng của các hệ thống thực tế.
- Nhiễu thêm vào là nhiễu trắng đơn giản, chưa xét đến các yếu tố gây nhiễu khác.
- Phần DCT sử dụng kỹ thuật giữ tần số thấp nhưng vẫn chưa sử dụng được phương pháp nén nâng cao như lượng tử hóa.
- Chưa kết hợp với phần cứng.
- Báo cáo mang nặng tính học thuật vừa là ưu điểm, vừa là nhược điểm khi thiếu đi tính ứng dụng.

6.2.2. Đề xuất hướng phát triển

Nhóm nhận thấy đề tài có thể tiếp tục phát triển trong tương lai theo các hướng sau:

- Sử dụng đáp ứng xung h(n) thực tế được đo từ thiết bị vật lý để bám sát thực tế, tăng tính ứng dụng.
- Kết nối mô hình với phần cứng thực tế như các vi điều khiển ESP32, STM32, Arduino Uno để điều khiển thiết bị dựa trên mức độ hồi phục tín hiệu.
- Mở rộng hệ thống sang tín hiệu âm thanh và hình ảnh thực tế để ứng dụng trong truyền thông.
- Sử dụng thêm nhiều phép biến đổi khác như DFT, FFT,…

6.3. Tổng kết

Qua báo cáo lần này, nhóm đã tích lũy thêm được nhiều kiến thức về xử lý tín hiệu số, rèn luyện được tư duy phân tích, thiết kế và đánh giá hệ thông theo hướng học thuật kết hợp thực tiễn. Việc kết hợp giữa lý thuyết biến đổi z, DCT cùng với chỉ số đánh giá MSE cho thấy khả năng áp dụng các công cụ toán học vào việc giải quyết bài toán thực tế một cách hệ thống, logic và có cơ sở định lượng rõ ràng.Bên cạnh đó, nhóm đã sử dụng tốt các công cụ hỗ trợ như

trí tuệ nhân tạo (AI), các bài báo khoa học, video học thuật, Canva và vẫn luôn giữ tinh thần " nhận cần câu chứ không nhận con cá".

Người thực hiện báo cáo

Đặng Bá Trần Trung

TÀI LIỆU THAM KHẢO

[1] Anaconda, Inc. (n.d.). Anaconda documentation.

https://www.anaconda.com/

[2] The Quantitative Methods for Psychology. (2020). Anaconda as a toolkit for psychological research. The Quantitative Methods for Psychology, 16(5), 1–9

https://www.tqmp.org/SpecialIssues/vol16-5/S003/S003.pdf

[3] Project Jupyter. (n.d.). Jupyter: open-source tools for interactive computing.

https://jupyter.org/

[4] IOP Conference Series: Materials Science and Engineering. (2019). Comparative study of image compression using DCT and DWT techniques.

https://iopscience.iop.org/article/10.1088/1757-899X/557/1/012027/pdf

[5] Al-Khafaji, A. S. (n.d.). Image Compression Using DCT and DWT.

https://coehuman.uodiyala.edu.iq/uploads/Coehuman%20library%20pdf/%D9%83%D8%AA%D8%A8%20%D8%A7%D9%84%D8%B1%D9%8A%D8%A7%D8%B6%D9%8A%D8%A7%D8%AA%20Mathematics%20books/Wavelets/25%20(2).pdf

[6] Hassan, M., & Mahmood, N. H. (2018). Image Quality Assessment and MSE/PSNR Performance Evaluation. SCIRP – Scientific Research Publishing.

https://www.scirp.org/journal/paperinformation?paperid=90911